Self-supervised Trajectory Representation Learning with Multi-scale Spatio-temporal Feature Exploration

Hong Xia[1], Xiao Zhang[1], Yuan Cao[1], Lei Cao[2], Yanwei Yu[1*], Junyu Dong[1]

[1]Faculty of Information Science and Engineering, Ocean University of China, Qingdao, China
[2]Department of Computer Science, The University of Arizona, Tucson, USA
{xhong, zhangxiao4549}@stu.ouc.edu.cn, lcao@csail.mit.edu,
{cy8661, yuyanwei, dongjunyu}@ouc.edu.cn

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00064

***Abstract*—Trajectory representation learning transforms the complex spatio-temporal features of trajectories into a dense, low-dimensional embedding, which supports various downstream analytics tasks such as trajectory classification, travel time estimation, and similar trajectory search. Existing trajectory representation learning methods treat trajectories merely as general point sequences and use sequence models to learn the correlations between points. However, the complex spatio-temporal features of trajectories are multi-scale, meaning they are not only reflected in the correlations between trajectory points but also in the correlations between trajectory segments. Moreover, most existing methods do not sufficiently capture the multi-faceted temporal features within trajectories. To fill these gaps, we propose a novel self-supervised Trajectory Representation Learning model with multi-scale spatio-temporal features exploration called *TrajRL*. Specifically, we utilize trajectory augmentation to generate two views to achieve self-supervised pre-training exploiting multiple self-supervisory signals. In each view, we can learn the multi-scale spatio-temporal correlations both within and between road segments and road segment sequences in trajectories through the proposed multi-scale trajectory encoder. Additionally, we perform multi-faceted temporal information encoding, especially leveraging time intervals to learn multi-scale context-aware time patterns within the trajectories. Extensive experiments demonstrate the superiority of our TrajRL as compared to state-of-the-art baselines on two real-world datasets across various downstream tasks. The source code of our model is available at https://github.com/Xfc30/TrajRL.**

I. Introduction

With the development of GPS-enabled devices and location-based services including map services and location-based social networks, a large amount of trajectory data has been generated. As one of the most typical types of spatio-temporal data, trajectory data provides valuable insights into the movement behaviors of people and objects. The analysis and management of trajectory data [1] [2] play a vital role in various fields, such as urban planning [3] [4] [5] [6], intelligent transportation systems [7] [8] [9] [10] [11], and infectious disease prevention [12].

*Corresponding author: Yanwei Yu.

Trajectory representation learning (TRL) aims to map trajectories from a sparse high-dimensional space into meaningful and dense low-dimensional representations that can be used for a variety of tasks.

Early studies on TRL are based on sequential models designed for a specific downstream task and trained in an end-to-end manner [13], [14]. These methods, while intuitive, suffer from limitations such as scalability and efficiency [15] [16]. Specifically, the representations learned are tailored to specific tasks, rendering them less applicable across different tasks. Moreover, the end-to-end embedding approach relies on the availability of extensive task labels to achieve satisfactory outcomes. To address these issues, self-supervised TRL has become increasingly popular. Seq2seq-based methods are firstly proposed, which are trained with reconstructive loss [17] [14] [18]. In order to avoid redundancy and noise in fine-grained GPS trajectories, some methods use road segment sequences as a substitute for GPS trajectories [19] [20] [21]. Recently, with the rise of graph neural networks, two-stage methods [21] have been proposed. These methods first learn the embeddings for road segments by graph neural network with road network graph as input. Then, the trajectory representations are learned by inputting the road segment representations into sequence-based deep learning models with self-supervised tasks for training.

Nevertheless, there are still two main challenges to be solved in self-supervised TRL. (1) **Poor generalization across multiple downstream tasks.** Since each downstream task has its unique characteristics, existing TRL methods often perform well on some tasks but poorly on others. The primary reason for this failure is that existing methods treat trajectories as general point sequences and apply RNN or Transformer to capture correlations between points. However, the spatio-temporal features and travel patterns embedded in trajectories are not only present between individual trajectory points but also between trajectory segments and between different trajectories. In other words, trajectories inherently possess

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00064

Authorized licensed use limited to: Ocean University of China. Downloaded on September 27,2026 at 02:16:00 UTC from IEEE Xplore. Restrictions apply.

multi-scale features, which previous work has overlooked by focusing only on point-level or segment-level features. Reconsidering real-world downstream tasks for TRL, different tasks are often more related to trajectory features at various scales. For example, destination prediction is strongly correlated with the features of individual points in a trajectory, while the trajectory similarity measure is more related to the overall structure of the trajectory. Therefore, developing a trajectory representation learning method that captures multi-scale spatio-temporal features is essential for acquiring high-quality representations and enhancing generalization across various downstream tasks.

(2) **Inadequate temporal information learning.** Most of the TRL methods only treat the trajectories as ordered sequences of locations and focus on mining the spatial information to learn the trajectory representations. However, as a type of spatio-temporal data, the temporal information should also be captured. To the best of our knowledge, prior to our work, only Trembr [22] and START [21] consider temporal information in self-supervised TRL. Trembr is a seq2seq model whose decoder considers the timestamps of trajectory points, while START incorporates temporal periodic encoding in the road segment representation.

In addition to temporal periodicity, the time intervals between trajectory points contain additional valuable information. Even in the spatial domain, two identical trajectories may have significantly different time interval distributions. This may reflect the traffic conditions of the road network when the trajectories are generated and the habitual movement patterns of the individuals or objects that produce them. For example, the congestion level in urban traffic and individual driving habits are both reflected in the time interval information. Therefore, more comprehensive temporal information extraction needs to be integrated into trajectory representation learning methods.

To address the aforementioned challenges, we propose a novel self-supervised **Traj**ectory **R**epresentation **L**earning with multi-scale spatio-temporal feature exploration, called **TrajRL**. The proposed method explores multi-faceted temporal information mining and aims to learn multi-scale trajectory features on two self-supervised tasks. To be specific, we leverage road network matching and trajectory augmentation to produce two views to build a self-supervised pre-training framework. In each view, we first use GAT to learn the initial embeddings of road segments that implicit road network connectivity and road segment features. Then, we use the proposed multi-scale trajectory encoder to learn the spatio-temporal correlations at different scales in road-network constrained trajectories.

Specifically, the proposed multi-scale trajectory encoder divides the trajectory into multiple trajectory segments and learns the correlations both within and between these segments. Through layer-wise aggregation, it effectively captures trajectory features across multiple scales, from fine to coarse. Furthermore, the proposed multi-faceted time encoder performs both temporal periodicity encoding and time interval encoding. Additionally, we further divide time intervals into multiple scales, capturing context-aware temporal patterns while avoiding the impact of fine-grained noise.

To pre-train our TrajRL across multiple scales, we employ segment-level trajectory recovery and trajectory contrastive learning tasks. The former provides fine-grained self-supervised signals, while the latter offers coarse-grained self-supervised signals.

Key contributions of this paper are summarized as follows:

- We propose a novel self-supervised trajectory representation learning framework, which sufficiently explores the multi-scale spatio-temporal features in trajectory data to learn generic trajectory representations for various downstream mining tasks. To pre-train our TrajRL, we apply two self-supervised tasks that provide self-supervised signals at two different scales.
- We construct a multi-faceted time encoder that effectively learns both temporal period regularities and multi-scale context-aware temporal patterns in trajectories through the designed time period encoder and time interval encoder, respectively.
- We design a multi-scale trajectory encoder, which simultaneously captures the spatio-temporal correlations within and between road segments and road segment sequences in trajectories through the designed InPatch encoder and CrossPatch encoder at different scales.
- The experiments conducted on two real-world datasets across three downstream tasks verify that our model achieves the best performance and demonstrates certain efficiency advantages.

II. Related work

We will briefly review related work on TRL from two aspects: learning spatio-temporal information of trajectories and self-supervised learning for trajectory representation.

A. Learning Spatio-temporal Information of Trajectories

1) Trajectory spatial information utilization: Trajectory representation learning aims to map raw trajectories into a universal high-dimensional latent vector space, applicable across various domains of trajectory data mining such as similar trajectory search, travel time estimation, and anomaly detection [23] [24] [25] [26] [27]. Early trajectory representation learning focuses on free-space trajectories, treating them as simple sequential data, thus failing to retain spatio-temporal information within embeddings. [14] [18] [28] typically use GPS trajectories as model inputs, naturally overlooking interactions between GPS trajectory data and real road networks.

In recent years, several two-stage methods are proposed. The first step is to learn embeddings of road segments. Toast [19] trains skip-gram [29] models to obtain embeddings of road networks by introducing auxiliary tasks to predict traffic-related contextual features (e.g., speed limits) in a self-supervised manner. This approach can struggle to perform effectively when road contextual information is unavailable. A more robust alternative could be Graph Neural Networks

Authorized licensed use limited to: Ocean University of China. Downloaded on September 27,2026 at 02:16:00 UTC from IEEE Xplore. Restrictions apply.

(GNNs), which leverage the structural information of road networks more effectively. JCLRNT [30] employs the more powerful GAT [31] for handling road network information. Trembr [22] utilizes a multi-task learning framework that simultaneously considers the co-occurrence and type relationships between road segments to learn their embeddings. START [21] leverages the properties of the road network and considers vehicle transfer probabilities to learn robust and general representations. The second step is to encode the trajectory as a sequence. To obtain trajectory embeddings, the above methods almost use basic sequence models such as RNNs or Transformers. To enhance the model's scalability, LightPath [32] proposes performing sparse operations on trajectories before inputting into the Transformer encoder. Specifically, recent work MM-STGED [33] models trajectories directly as graphs. In this study, we consider the relationships between trajectory segments at different scales and explore the design of a multi-scale trajectory encoder.

2) Trajectory temporal information utilization: Trembr introduces time information in the decoder for trajectory reconstruction, but it does not explicitly model time information. START extracts the time periodicity through time embedding, and it uses time information during self-attention weight computation in the transformer encoder to enhance the information aggregation. We design a multi-faceted time encoder that explicitly models both time periodicity and time intervals.

B. Self-supervised Learning for Trajectory Representation

Early work on trajectory representation learning has predominantly relied on labeled data [34] [10] [35] [36]for end-to-end training, which introduces two major challenges. First, the embeddings generated by these models are often biased and lack generalizability, making it difficult to apply them to other tasks [37] [38] [39]. Secondly, obtaining true labels for the data is inherently costly.

To address these issues, there is a growing interest in pretraining trajectory embeddings with self-supervised learning. Self-supervised learning utilizes labels generated from the data itself or auxiliary tasks to train, enabling models to acquire general and robust embedding even in the absence of labels [40] [41]. In the field of TRL, autoencoders based on sequence models are initially widely adopted, learning trajectory representations based on reconstruction loss. T2vec [14] employs an RNN-based seq2seq model, maximizing the conditional probability of reconstructing high-resolution trajectories from low-sampling-rate ones to learn trajectory latent representations. Traj2vec [18] considers spatio-temporal transformations of trajectory data, utilizing differences in a set of attributes between consecutive trajectory points as input features. Additionally, Trembr [22], as mentioned earlier, aims to learn representations by simultaneously reconstructing the road segments and timestamps of the input trajectories.

Toast [19] and PIM [20] use masked language model (MLM) [42] prediction with Transformer and mutual information maximization with RNN, respectively, as self-supervised tasks to generate general trajectory representations. TrajCL [28] employs a contrastive learning approach to learning more discriminative feature representations. Similarly, START [21] using contrastive learning to train the encoder, an additional trajectory recovery pretext task is incorporated.

To improve the generalization of trajectory embeddings, MMTEC [43] uses the maximum entropy principle to ensure consistency between two views of the same trajectory, thereby reducing pretraining bias when using different self-supervised tasks. We employ two self-supervised tasks that provide self-supervised signals at two different scales. More importantly, such self-supervised pretraining methods can mutually enhance our multi-scale trajectory representation learning model, thereby obtaining more generalized representations.

III. Preliminaries

In this section, we first introduce the basic preliminaries used in this paper, and then we formally define the problem of trajectory representation learning.

TABLE I: Summary of frequently used notations.

Notations	Definition
p	raw GPS trajectory point.
v	road segment.
t	the visiting time of a point or a road sgement in a trajectory
$\mathcal{T}$	a road-network constrained trajectory
n	the length of a trajectory
h_i^l	the representation embedding for the i-th node in graph $\mathcal{G}$ in the l-th layer of GAT.
Δ	the time interval series of a trajectory
$T_{t_i}^P$	the time period embedding of time t_i
$t_{S(i,j)}$	the embedding of i-th time interval scaled by j-th scale coefficients
$T_{t_i}^S$	the multi-scaled time interval embedding of time t_i
x_i	the embedding of a road segment in a given trajectory.
ω	the window size in the patch mask layer.
e_i	the embedding of i-th patch obtained by InPatch encoder
$z^{(l)}$	the l-th scaled trajectory embedding of trajectory $\mathcal{T}$
z	the embedding of trajectory $\mathcal{T}$

Definition 1 (Road Network)**.** *A road network is represented as a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{F})$, where $\mathcal{V}$ is the set of vertices, each vertex $v_i \in \mathcal{V}$ representing a road segment. $\mathcal{E}$ is the set of edges, each $e_{i,j} = (v_i, v_j)$ representing the intersection between road segments v_i and v_j. $\mathcal{F} \in \mathbb{R}^{|\mathcal{V}| \times f}$ is the feature matrix of road segments, where f is the feature dimension.*

Definition 2 (GPS-based Trajectory)**.** *A GPS-based trajectory $\mathcal{T}^{raw} =< p_1, p_2, \ldots, p_n >$ is a sequence of spatial-temporal sample points ordered in time, where each sample point $p_i = \langle lat_i, lon_i, t_i \rangle$ is a triplet consisting of latitude, longitude, and sampling time.*

Definition 3 (Road-network Constrained Trajectory)**.** *A road-network constrained trajectory $\mathcal{T} =< r_1, r_2, \ldots, r_n >$ is a sequence of adjacent road segments ordered in time, where $r_i = \langle v_i, t_i, \mu_i \rangle$ and $v_i \in \mathcal{V}$ presents the i-th road segment in the sequence, t_i is the visit time for v_i, and $t_i < t_{i+1}$, $\mu_i \in [0, 1]$ representing the ratio of moving distance along the road segment.*

Authorized licensed use limited to: Ocean University of China. Downloaded on September 27,2026 at 02:16:00 UTC from IEEE Xplore. Restrictions apply.

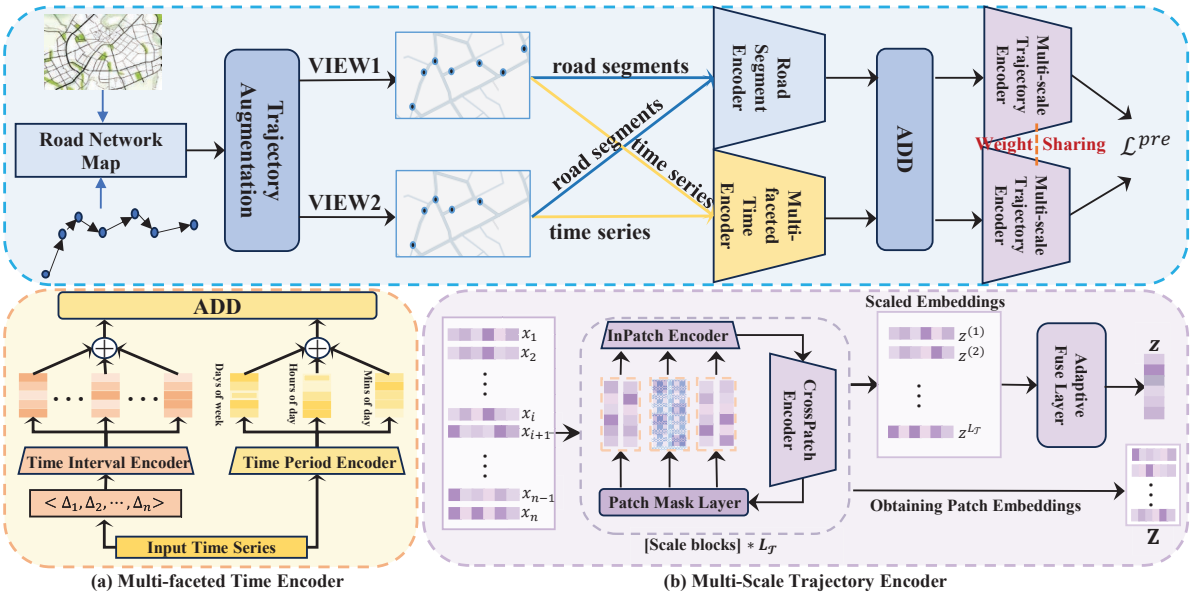


Fig. 1: Overall Framework of the Proposed TrajRL.

In this work, we mainly focus on the road-network constrained trajectories. For simplicity, we use trajectory to refer to road-network constrained trajectory in the following.

Next, we formally define our studied problem of trajectory representation learning as follows:

Problem 1. *Given a road network $\mathcal{G}$ and a dataset of trajectories $\mathcal{D} = \{\mathcal{T}_i\}_{i=1}^{|\mathcal{D}|}$, the trajectory representation learning aims to learn a generic low-dimensional vector $h_i \in \mathbb{R}^d$ for each trajectory $\mathcal{T}_i \in \mathcal{D}$ that preserves the spatio-temporal correlations.*

In this work, our goal is to acquire a robust and generic representation vector for each trajectory that achieves optimal performance on multiple downstream tasks.

IV. Methodology

In this section, we present the details of our proposed **TrajRL** framework, as illustrated in Figure 1. Our proposed TrajRL consists of five key components: (1) *Road-network Matching and Trajectory Augmentation*, (2) *Road Segment Encoder*, (3) *Multi-faceted Time Encoder*, (4) *Multi-scale Trajectory Encoder*, and (5) *Self-supervised Pre-training*.

A. Road-network Matching and Trajectory Augmentation

First, we perform *map matching* [44] to map the points in the raw trajectories (*i.e.*, GPS-based trajectories) with road segments to get the road-network constrained trajectories. Specifically, GPS-based trajectories are converted into sequences of adjective road segments. For moving objects in the city, especially vehicles, such trajectory preprocessing is consistent with the actual situation in urban traffic scenarios.

Then, to capture the global and local features of trajectories, we adopt a contrastive learning framework based on trajectory augmentation. Through data augmentation, for each trajectory, we can obtain two different views, and then use the comprehensive trajectory encoder introduced later to learn aligned trajectory representations for self-supervised pre-training.

B. Road Segment Encoder

Given the graph structure of the road network, we use a graph neural network to learn the representations of road segments. On one hand, real trajectories are constrained by the topology of the road network. On the other hand, the attributes of road segments, such as the number of lanes and maximum speed, significantly influence the driver's route planning. Therefore, we learn the representations of road segments from both their static attributes and the topology of the road network.

First, we consider four features including road segment type, length, number of lanes, and maximum travel speed for each road segment, as well as the out-degree and in-degree within the road network. We perform normalization on continuous feature values and one-hot encoding on discrete feature values. All encoded features are concatenated into the initial embedding of the road segment, denoted as $\vec{v}_i \in \mathbb{R}^d$ for i-th road segment.

To flexibly integrate the embeddings of neighboring nodes, we adopt GAT [31] for message passing within the road network. The nodes in the graph $\mathcal{G}$ aggregate information from their neighboring nodes as:

$$h_i^{(l+1)} = \text{ELU}\left(\sum_{j \in \mathcal{N}_i} \alpha_{ij}^{(l)} h_j^{(l)} \mathbf{W}_1^{(l)}\right), \tag{1}$$

where $h_i^{(0)} = \vec{v}_i$, $\mathbf{W_1}^{(l)} \in \mathbb{R}^{d \times d}$ is learnable parameters for the l-th layer, $\mathcal{N}_i$ is the set of neighbors of v_i in graph $\mathcal{G}$, ELU

Authorized licensed use limited to: Ocean University of China. Downloaded on September 27,2026 at 02:16:00 UTC from IEEE Xplore. Restrictions apply.

is the Exponential Linear Unit activation function, and $\alpha_{ij}^{(l)}$ is the attention weight at the l-th layer.

Inspired by [21], we also use the transition probability between road segments in historical data to augment the attention weights. The attention weight calculation for the l-th layer is as follows:

$$\begin{aligned} \alpha_{ij}^{(l)} &= \frac{\exp(\text{LeakyReLU}(e_{ij}^{(l)}))}{\sum_{k\in\mathcal{N}_i}\exp(\text{LeakyReLU}(e_{ik}^{(l)}))}, \\ e_{ij}^{(l)} &= (h_i^{(l)}\mathbf{W}_2^{(l)} + h_j^{(l)}\mathbf{W}_3^{(l)} + p_{ij}\mathbf{W}_4^{(l)})\mathbf{W}_5^{(l)\mathsf{T}}, \\ p_{i,j} &= \text{count}(v_i \to v_j)/\text{count}(v_i), \end{aligned} \tag{2}$$

where $h_i^{(l)}, h_j^{(l)} \in \mathbb{R}^{d_l}$ are the embeddings of road segments v_i and v_j in the l-th layer, $\mathbf{W}_2^{(l)}, \mathbf{W}_3^{(l)} \in \mathbb{R}^{d\times d}$, $\mathbf{W}_4^{(l)}, \mathbf{W}_5^{(l)} \in \mathbb{R}^{1\times d}$ are learnable parameters, d is the hidden vector dimension in GAT, LeakyReLU is the activation function, and $p_{i,j}$ is the transition probability between v_i and v_j.

The multi-head attention mechanism can be used to further strengthen the above node information aggregation. Finally, the road segment representation is obtained by concatenating the outputs of multiple heads.

C. Multi-faceted Time Encoder

The time sequence data within trajectories is nearly as crucial as the corresponding spatial sequence data, yet prior studies do not give adequate consideration to this aspect. To capture the various time factors in trajectories, we design a multi-faceted time encoder, as shown in the lower left of Figure 1. This module consists of a time period encoder and a time interval encoder.

1) Time Period Encoder: We first propose a time period encoder to capture the time-periodic patterns of urban traffic from the trajectory time series. To extract weekly, daily, and hourly periodicities, we extract the day of the week, the hour of the day, and the minute of the day, and transform them into the day-of-week index (1 to 7), hour-of-day index (1 to 24) and minute-of-day index (1-1440), respectively. For each visit timestamp t_i, we use three embeddings $t_i^d \in \mathbb{R}^d$, $t_i^h \in \mathbb{R}^d$, and $t_i^m \in \mathbb{R}^d$ to present these three periodic patterns. The time period embedding $T_{t_i}^P$ of time t_i is:

$$T_{t_i}^P = t_i^d + t_i^h + t_i^m. \tag{3}$$

2) Time Interval Encoder: Clearly, the periodic encoding of one road segment in a trajectory is independent of other road segments within the same trajectory, *i.e.*, context-independent, indicating an inability to represent the patterns of change in the time dimension of trajectories.

To address this issue, we next design a multi-scale time interval encoder. Specifically, we first obtain the corresponding time interval series for the time series of a trajectory $< t_1, t_2, \ldots t_n >$ as:

$$\begin{aligned} \Delta &= < \Delta_1, \Delta_2, \ldots, \Delta_n >, \\ \Delta_i &= \begin{cases} 0 & \text{if } i = 1 \\ t_i - t_{i-1} & \text{if } i > 1 \end{cases}. \end{aligned} \tag{4}$$

Notice that we set an upper bound for time intervals to ignore infrequent large time interval values to avoid noise interference. Then, we select λ scale coefficients $\{c_1, c_2, \ldots, c_\lambda\}$ to scale the value Δ_i in the time interval series Δ. For each scale c_j, we define a scaling function as:

$$S(i,j) = \lfloor \Delta_i \times \frac{1}{c_j} \rfloor. \tag{5}$$

Then, all scaled time interval series undergo word embedding [45]. A time interval series scaled by coefficient c_j is then encoded as n embeddings:

$$< t_{S(1,j)}, t_{S(2,j)}, \ldots, t_{S(n,j)} >, \tag{6}$$

where $t_{S(i,j)} \in \mathbb{R}^d$ is the corresponding scaled time interval embedding of $S(i,j)$.

Eventually, by fusing multiple scaled time interval embeddings, we can obtain the multi-scale time interval embedding for each time t_i:

$$T_{t_i}^S = \sum_{j=1}^{\lambda} t_{S(i,j)}. \tag{7}$$

It can be seen from the above that the time interval encoding of each road segment in a trajectory constructs the association between the road segment and the previous road segment, so the time interval embedding sequence represents the change pattern of the trajectory in the time dimension.

After obtaining the time period embedding and time interval embedding, we add the two as the time embedding of time t_i:

$$T_{t_i} = T_{t_i}^P + T_{t_i}^S. \tag{8}$$

For a road segment v_i in a given trajectory $\mathcal{T}$, its representation aggregates spatial embedding, time embedding, and position encoding:

$$x_i = h_i + T_{t_i} + pe_i, \tag{9}$$

where pe_i denotes the position encoding in $\mathcal{T}$ used in the subsequent module.

D. Multi-Scale Trajectory Encoder

To better learn rich traffic patterns and travel semantics in road-network constrained trajectories, we design a multi-scale trajectory encoder to explicitly model and capture the associations and dependencies between road segments in the trajectories.

Our multi-scale trajectory encoder consists of $L_\mathcal{T}$ layers of scale blocks. Each block performs three steps: dividing the trajectory into segments, encoding within the trajectory segment, and encoding between the trajectory segments. The upper-level scale blocks use the output of the lower-level blocks as input to further learn the travel patterns at larger scales in the trajectory. By stacking multiple scale blocks, we achieve multi-scale spatio-temporal information capturing, thereby obtaining a more realistic trajectory representation that serves downstream tasks.

Next, we present the forward propagation process for each scale block.

Authorized licensed use limited to: Ocean University of China. Downloaded on September 27,2026 at 02:16:00 UTC from IEEE Xplore. Restrictions apply.

1) Trajectory Patch: Inspired by the time series partitioning method proposed by *PatchTST* [46], we divide the trajectory into non-overlapped patches. The window size is denoted as ω, then for the input trajectory $\mathcal{T} =< r_1, r_2, \ldots, r_n >$ for the first layer of scale block, the patching operation generates a sequence of patches:

$$< s_1, s_2, \ldots, s_N >, \tag{10}$$

where s_i is a patch, representing a sequence of road segments in the trajectory within a window, and $N = \lceil \frac{n}{\omega} \rceil$. For trajectories whose length is less than n, we pad these trajectories with padding markers before input into the multi-scale trajectory encoder to ensure that they are of the same length.

2) InPatch Encoder: InPatch encoder is proposed to capture the correlation between road segments within a patch. Given that a patch is a sequence of road segments in a trajectory with a length of window size, we select GRU [47] as the encoder which is computationally efficient.

Let $\mathbf{X}_i \in \mathbb{R}^{\omega \times d}$ be the initial embedding matrix of the i-th patch s_i, which is a matrix composed of the representation vectors of ω road segments in the patch. The formal formula of GRU is as follows:

$$\mathbf{H}_i = \text{GRU}(\mathbf{X}_i, \mathbf{H}_0), \tag{11}$$

where $\mathbf{H}_0$ is the randomly initialized initial hidden state embedding matrix, and $\mathbf{H}_i$ is the output of GRU. The embedding vector $e_i \in \mathbb{R}^d$ of each patch is obtained by a mean pooling layer:

$$e_i = \textit{MeanPool}(\mathbf{H}_i). \tag{12}$$

Through the design of the InPatch encoder, we can not only capture the travel patterns between the road segments within the patch, but also learn the overall embedding of each Patch. This allows us to view each patch as a complete entity and effectively capture more detailed spatio-temporal semantics between patches.

3) CrossPatch Encoder: We use a vanilla Transformer encoder as the CrossPatch encoder to capture the correlation between trajectory segments (*i.e.*, patches). Now, a trajectory is represented as an ordered sequence of multiple patches $< e_1, e_2, \ldots, e_N >$. Let $\mathbf{E} \in \mathbb{R}^{N \times d}$ denote the embedding matrix of these patches.

These patches are then fed into a multi-head Transformer module to capture the correlation between patches:

$$\mathbf{Z} = \textit{MultiHeadTransformer}(\mathbf{E}), \tag{13}$$

where $\mathbf{Z} \in \mathbb{R}^{N \times d}$ is the scaled embeddings of patches and will be fed into the next layer of scale block.

With the use of patches, the length of the input sequence can be reduced from n to $\lceil \frac{n}{\omega} \rceil$. This greatly reduces the computational complexity and memory usage of the attention mechanism. Constrained on training time and GPU memory, this also allows our TrajRL to learn representations for longer trajectories.

After stacking $L_\mathcal{T}$ layers of scale blocks, we obtain all blocks' output as $\{\mathbf{Z}^{(1)}, \mathbf{Z}^{(2)}, \ldots, \mathbf{Z}^{(L_\mathcal{T})}\}$, where $\mathbf{Z}^{(i)}$ is the output of the i-th block layer. Remember that the rows in the embedding matrices output by different scale blocks are different. This is because the InPatch encoder not only learns patch representation but also plays the role of merging the representations of adjacent patches. Through this merging mechanism, we can continuously learn larger-scale representations from smaller-scale representations. To extract the overall representation $\mathbf{z} \in \mathbb{R}^d$ of the whole trajectory, we first mean pool the scaled patch embeddings to obtain the trajectory embedding at each scale. The l-th scaled trajectory embedding is:

$$z^{(l)} = \textit{MeanPool}(\mathbf{Z}^{(l)}). \tag{14}$$

Then, the scaled embeddings are fused by adaptively adjusting weights in the multi-scale trajectory encoder:

$$z = \sum_{l=0}^{L_\mathcal{T}} \beta^{(l)} \mathbf{z}^{(l)}, \qquad \sum_{l=0}^{L_\mathcal{T}} \beta^{(l)} = 1 \tag{15}$$

where $\beta^{(l)}$ is the learnable weight coefficient.

E. Self-supervised Pre-training

Our work aims to learn generalized trajectory representations that can serve multiple downstream tasks. Therefore, we need to pre-train our TrajRL using the self-supervisory signal of the trajectory itself instead of any supervised signal. To promote our TrajRL to capture multi-scale spatio-temporal features, we simultaneously employ two self-supervised tasks that provide self-supervised signals from different scales.

1) Patch-level Trajectory Recovery: Trajectory recovery is a self-supervised task similar to MLM [42]. In this work, trajectories are represented as sequences of continuous road segments, so if some road segments in trajectories are masked, the model can easily infer these road segments because the connection relationship between these road segments is implied in the road network. In the proposed multi-scale trajectory encoder, patch operation also can avoid this problem. Specifically, we use masked patches instead of masked road segments. Before feeding the trajectories into the multi-scale trajectory encoder, we perform a patch and mask operation on the trajectories with window size ω. Note that we use an independent and identical uniform distribution to generate the mask probability of each patch and ensure that the masked patches are not continuous. To facilitate the evaluation of the accuracy of the recovered trajectories, we add a scale block whose window size is 1 to obtain the trajectory embedding at the scale of a road segment. The window size in other scale blocks is still ω.

After obtaining the embedding $\mathbf{Z}^{(1)} \in \mathbb{R}^{|\mathcal{V}| \times d}$ of a trajectory $\mathcal{T}$ with mask markers, we use a linear classifier to predict the road segments in the masked patches:

$$\hat{\mathbf{Z}}^{(1)} = \mathbf{Z}^{(1)} \mathbf{W}_m + b_m, \tag{16}$$

where $\mathbf{W}_m \in \mathbb{R}^{d \times |\mathcal{V}|}$ and $b_m \in \mathbb{R}^{|\mathcal{V}|}$ are learnable parameters.

Authorized licensed use limited to: Ocean University of China. Downloaded on September 27,2026 at 02:16:00 UTC from IEEE Xplore. Restrictions apply.

Finally, we use the cross-entropy loss between masked roads and predicted values in trajectory $\mathcal{T}$ as the loss function:

$$\mathcal{L}_{\mathcal{T}}^{\text{mask}} = -\frac{1}{|\mathcal{M}|}\sum_{i=1}^{|\mathcal{M}|}\log\frac{\exp(\hat{\mathbf{Z}}_i^{(1)})}{\sum_{j=1}^{|\mathcal{V}|}\exp(\hat{\mathbf{Z}}_j^{(1)})}, \tag{17}$$

where $\mathcal{M}$ is the set of road segments in the masked patches. For a batch of trajectories, we average all trajectories' losses as the loss $\mathcal{L}^{mask}$.

2) Trajectory Contrastive Learning: Trajectory recovery is dedicated to mining the correlation between trajectory segments from contextual information, so it is local self-supervised signals. To improve the model's ability to learn global trajectory representations, we also introduce contrastive learning which provides self-supervised signals from a global perspective.

Trajectory Augmentation. Data augmentation creates different views of input data such that the model can learn common and distinguishing features from the views. [28] explores data augmentation methods for GPS-based trajectories, but they are not applicable to road-network constrained trajectory data. In this work, we further extend the existing trajectory augmentation methods for road-network constrained trajectories. In particular, based on previous work [28], we select two effective trajectory augmentation methods to produce appropriate trajectory views.

- *Road segment shifting*: Given a trajectory $\mathcal{T}$, road segment shifting randomly alters the road segments or the corresponding timestamps within the trajectory. Specifically, for the randomly selected road segment, road segment shifting randomly changes the current road segment to one of its one-hop or two-hop neighbors in the road network (excluding its preceding and following adjacent road segments in the trajectory). Temporal shifting performs a random perturbation to the corresponding timestamp.
- *Trajectory truncating*: Given a trajectory $\mathcal{T}$, trajectory truncating cuts a prefix or a suffix from $\mathcal{T}$ According to a given ratio, and keeps the rest as a new view $\tilde{\mathcal{T}}$.

We use the InfoNCE [48] loss for model training. Two different views of the same trajectory constitute a positive pair and any pair of views from different trajectories constitutes a negative pair. The contrastive training loss for a positive pair (i, j) is defined as:

$$\begin{aligned}\mathcal{L}_{i,j}^{cl} &= -\log\frac{\exp(\text{Sim}(\mathbf{z}_i,\mathbf{z}_j)/\tau)}{\sum_{[k=1]\wedge[k\neq i]}^{2N_b}\exp(\text{Sim}(\mathbf{z}_i,\mathbf{z}_k)/\tau)},\\ \text{Sim}(\mathbf{z}_i,\mathbf{z}_j) &= \frac{\mathbf{z}_i\cdot\mathbf{z}_j}{\|\mathbf{z}_i\|\|\mathbf{z}_j\|},\end{aligned} \tag{18}$$

where N_b is the batch size, τ is the temperature hyperparameter. We average all $2N_b$ trajectories' losses as the loss $\mathcal{L}^{cl}$.

We pre-train our TrajRL with the above two self-supervised tasks. The pre-training loss is the sum of trajectory recovery loss and contrastive loss:

$$\mathcal{L}^{pre} = \gamma\mathcal{L}^{mask} + (1-\gamma)\mathcal{L}^{cl}, \tag{19}$$

where γ is the hyperparameter as a weight to balance the two self-supervised tasks.

F. Time Complexity Analysis

The time complexity of TrajRL is determined by the self-attention computations in the scale block module. The time complexity of i-th scale block is $O\left(L_H d\sum_{i=1}^{l}\left(\frac{n}{\omega^{i-1}}\right)^2\right)$ where L_H is the layers of each multi-head transformer encoder, n is the length of a trajectory and ω is the window size in the multi-scale trajectory encoder. Since the InPatch encoder merges patches within a window, the number of layers $L_{\mathcal{T}}$ of scale blocks is $\lfloor\log_\omega n+1\rfloor$. Here, for the sake of simplicity, we ignore the rounding operations. By summing the time complexities of all scale blocks, we can derive the time complexity of TrajRL as $O\left(\sum_{i=1}^{L_\tau}L_H dn^2\left(\frac{1}{\omega^2}\right)^i\right)$, which is $O\left(L_H dn^2\cdot\frac{1-\frac{1}{n^2\omega^2}}{1-\frac{1}{\omega^2}}\right)$, approximating $O\left(\frac{L_H dn^2}{1-\frac{1}{\omega^2}}\right)$. Thus, it is comparable to the time complexity of the Transformer encoder.

G. Model Fine-tuning

We fine-tune TrajRL for downstream tasks using the simplest methods.

1) Trajectory Classification: We feed the trajectory representations into a simple linear classifier, which consists of a linear layer followed by a softmax activation function. The predicted value is $\hat{y}_i = softmax(FC(\mathbf{z}_i)) \in \mathbb{R}^C$. Then we fine-tune our TrajRL using cross-entropy loss:

$$\mathcal{L}^{cla} = -\frac{1}{N_{test}}\sum_{i=1}^{N_{test}}\sum_{c=1}^{C}y_{i,c}\log(\hat{y}_{i,c}), \tag{20}$$

where $y_{i,c}$ is the ground truth of i-th trajectory, N_{test} is the total number of trajectories in the test set, and C is the number of categories.

2) Trajectory Travel Time Estimation: Similarly, we feed the trajectory representations into a simple linear regression model to obtain the predicted value as $\hat{y}_i = FC(\mathbf{z}_i) \in \mathbb{R}$. Then we fine-tune our TrajRL using mean square error loss:

$$\mathcal{L}^{tte} = \frac{1}{N_{test}}\sum_{i=1}^{N_{test}}\|y_i-\hat{y}_i\|^2, \tag{21}$$

where y_i is the ground truth of i-th trajectory and N_{test} is the total number of trajectories in the test set.

3) Similar trajectory search: Given a query trajectory, this task is to find out the most similar trajectory in the database. Here, there is no fine-tuning, and we directly use the representation vector $\mathbf{z}_i$ obtained from our TrajRL for similarity computation. For detailed specifics, please refer to Section V-C2.

V. Experiments

In this section, we conduct extensive experiments to evaluate our TrajRL for trajectory representation learning. The experiment evaluation includes five parts:

- **Performance Comparison**. We compare the performance of our TrajRL with the state-of-the-art methods on two real datasets for three downstream tasks.

Authorized licensed use limited to: Ocean University of China. Downloaded on September 27,2026 at 02:16:00 UTC from IEEE Xplore. Restrictions apply.

- **Ablation Study**. We design the variants of TrajRL and compare their performance with the complete model to validate the effectiveness of each designed module.
- **Pre-training Effect Study**. We investigate the impact of self-supervised pre-training on model performance and convergence speed.
- **Parameter Sensitivity Analysis**. We evaluate the impact of different parameter settings on the performance of our TrajRL to verify its stability.
- **Efficiency Comparison**. We compare the inference time of TrajRL with all the baselines to demonstrate the superiority of TrajRL in terms of efficiency.

TABLE II: Statistics of the two datasets after preprocessing.

Dataset	Porto	BJ
Time span	07/01/2013-07/01/2014	11/01/2015-11/30/2015
#Trajectories	695,085	1,018,312
#Users	435	1,677
#Road Segments	10,903	38,479

A. Datasets

We conduct the experiments on two real-world large-scale trajectory datasets, Porto and BJ. Porto[1] is an open-source dataset containing 1.7 million taxi trajectories from Porto, Portugal, between July 2013 and June 2014. BJ is a newly released dataset by [21] that collected over 1 million taxi trajectories from Beijing in November 2015. We download map data of Porto from OpenStreetMap[2] to construct the road network. The OpenStreetMap data contains all road segments in the cities, the connection relationships between all road segments, and the features of road segments. For the BJ dataset, the authors map the trajectories to the road network, making it ready for direct use.

We partition the dataset into training, validation, and test sets chronologically. For Porto, We partition each month's data in a 7:2:1 ratio and combine the data from each month to form the training, validation, and test sets. For BJ, These three sets cover 17/6/7 days of data, respectively, with the final data ratio being approximately 7:1.5:1.5. Additionally, we set the maximum trajectory length to 128 road segments. The statistics of the two datasets after preprocessing are summarized in Table II.

B. Compared Methods

To evaluate the superiority of our TrajRL, we conclude comparisons with nine state-of-the-art pre-training trajectory representation learning methods. We select representative general models designed for multiple downstream tasks. Additionally, we choose several specialized models designed for a specific important downstream task that can be easily extended to the experimental setting in the paper.

[1]https://www.kaggle.com/c/pkdd-15-predict-taxi-service-trajectory-i
[2]https://www.openstreetmap.org

- **Traj2vec** [18] transforms each trajectory into a feature sequence by capturing spatial-temporal invariant characteristics and employs a sequence-to-sequence auto-encoder to learn trajectory embeddings.
- **T2vec** [14] pre-trains an RNN trajectory encoder based on a denoising anto-encoder, aiming to recover fine-grained trajectories given spares ones.
- **Trembr** [22] is an RNN auto-encoder to learn trajectory embeddings with both road segment and timestamp recovery tasks.
- **PIM** [20] uses node2vec to generate road segment representations of the static road network. It generates negative samples following the principles of curriculum learning and uses mutual information maximization to optimize the RNNs encoder unsupervisedly.
- **Toast** [19] employs node2vec with traffic context awareness to learn road segment representations and then trains a Transformer encoder with an MLM-based task and a discrimination task to generate trajectory embeddings.
- **JCLRNT** [30] develops a graph encoder and a trajectory encoder to model the representations of road segments and trajectories, respectively. These representations are organized to train the model with three contrastive tasks.
- **LightPath** [32] utilizes a transformer-based autoencoder to construct a sparse path encoder. Specifically, it employs relational reasoning and knowledge distillation frameworks to enhance the model's scalability.
- **TrajCL** [28] is state-of-the-art model for trajectory similarity computation. It uses contrastive learning to pre-train the proposed spatial and structural dual-feature self-attention-based trajectory encoder.
- **START** [21] proposes a time-aware transformer encoder and optimizes a GAT with the transitions between road segments. The model is jointly trained by the loss functions of contrastive learning and the MLM task.

C. Experimental Settings

1) Implementation details: We implement our TrajRL with PyTorch 1.12.1 and Libcity [49] framework which is a comprehensive and extensible library in the traffic prediction field. We run experiments on a machine with Ubuntu 20.04 and an NVIDIA GeForce 4090 GPU. We set the embedding dimensionality d to 256. For road segment encoder, we set the layers L to 3 and the attention heads to [8,16,1]. For the multi-faceted time encoder, we set the number of scale coefficients λ to 3 and the scale coefficients to $\{10, 30, 60\}$ for Porto and $\{15, 60, 150\}$ for BJ. For the multi-scale trajectory encoder, we set the layers L_H of multi-head attention module in each transformer encoder to 3, the number of attention heads to 16, the window size ω to 4, and the number of scale blocks $L_{\mathcal{T}}$ to 4. The temperature parameter τ in InforNCE loss is 0.05. The coefficient γ used to balance the loss is 0.5. We pre-train and fine-tune our TrajRL using optimizer Adam [50]. The batch size is 64, and the training epoch is 30. All baseline methods maintain the same embedding size as TrajRL, and other settings follow the default or recommended configurations

Authorized licensed use limited to: Ocean University of China. Downloaded on September 27,2026 at 02:16:00 UTC from IEEE Xplore. Restrictions apply.

TABLE III: Performance comparison of all models on Porto Dataset.

Method	Trajectory Classification			Travel Time Estimation			Most Similar Trajectory Search		
	Micro-F1 ↑	Macro-F1 ↑	Recall@5 ↑	MAE ↓	MAPE ↓	RMSE↓	MR ↓	HR@1 ↑	HR@5 ↑
traj2vec [IJCNN-17]	0.066	0.041	0.181	1.549	0.241	2.346	30.69	0.543	0.732
t2vec [ICDE-18]	0.064	0.053	0.189	1.521	0.237	2.324	11.54	0.769	0.858
Trembr [TIST-20]	0.071	0.049	0.192	1.480	0.226	2.164	4.635	0.846	0.929
PIM [IJCAL-21]	0.059	0.037	0.158	1.568	0.247	2.642	20.75	0.617	0.764
Toast [CIKM-21]	0.062	0.035	0.184	1.624	0.246	2.445	22.61	0.684	0.789
JCLRNT [CIKM-22]	0.069	0.046	0.191	1.647	0.238	2.334	5.017	0.852	0.931
LightPath [KDD-23]	0.068	0.049	0.180	1.561	0.242	2.362	13.26	0.607	0.788
TrajCL [ICDE-23]	0.056	0.033	0.148	1.960	0.269	2.963	5.461	0.834	0.915
START [ICDE-23]	0.084	0.061	0.238	1.374	0.219	2.185	1.927	0.921	0.974
TrajRL	**0.092**	**0.070**	**0.249**	**1.259**	**0.202**	**2.051**	**1.241**	**0.988**	**0.997**
Improvement	9.52%	14.75%	4.62%	8.36%	7.76%	6.13%	35.6%	8.36%	2.36%

The metric with "↑" means that a larger result is better, and the metric "↓" means that a smaller result is better.
The **bold** results are the best, and the underlined results are the second best. The time in travel time estimation is in minutes.

TABLE IV: Performance comparison of all models on BJ Dataset.

Method	Trajectory Classification			Travel Time Estimation			Most Similar Trajectory Search		
	ACC ↑	F1 ↑	AUC ↑	MAE ↓	MAPE ↓	RMSE↓	MR ↓	HR@1 ↑	HR@5 ↑
traj2vec [IJCNN-17]	0.802	0.841	0.869	10.14	0.399	57.28	10.26	0.573	0.758
t2vec [ICDE-18]	0.799	0.859	0.861	10.27	0.372	56.15	9.684	0.763	0.905
Trembr [TIST-20]	0.818	0.871	0.880	9.997	0.342	36.97	2.509	0.884	0.952
PIM [IJCAL-21]	0.803	0.874	0.866	11.56	0.486	35.78	26.75	0.731	0.874
Toast [CIKM-21]	0.810	0.870	0.871	10.69	0.350	57.41	29.53	0.611	0.746
JCLRNT [CIKM-22]	0.825	0.867	0.883	10.73	0.406	57.84	6.382	0.802	0.927
LightPath [KDD-23]	0.802	0.859	0.865	10.66	0.506	55.80	6.484	0.748	0.867
TrajCL [ICDE-23]	0.764	0.722	0.730	10.99	0.523	59.97	5.524	0.827	0.948
START [ICDE-23]	0.842	0.881	0.904	9.134	0.336	35.31	3.081	0.969	0.993
TrajRL	**0.867**	**0.914**	**0.939**	**8.807**	**0.298**	**33.15**	**2.140**	**0.982**	**0.998**
Improvement	2.96%	3.74%	3.87%	3.58%	11.31%	6.11%	14.7%	1.34%	0.53%

The metric with "↑" means that a larger result is better, and the metric "↓" means that a smaller result is better.
The **bold** results are the best, and the underlined results are the second best. The time in travel time estimation is in minutes.

from the original papers. Additionally, all methods are fine-tuned for the downstream tasks in a consistent manner, as described in Section IV-G. Since *Trembr* and *Toast* are not open-sourced, we refer experiment setting and results from [21] for a fair comparison.

2) Downstream task settings: In Section IV-G, we explain how our TrajRL is fine-tuned for three mining tasks: trajectory classification, travel time estimation, and similar trajectory search. These three downstream tasks represent typical classification, regression, and query tasks that directly utilize learned representations. Trajectory classification is a typical classification task that represents several important tasks, such as linking trajectory users, determining whether a taxi is carrying a passenger, and identifying the mode of transportation. Travel time estimation is a typical regression task with significant applications in navigation software. Most similar trajectory querying is an important downstream task in trajectory similarity learning and serves as the foundation for constructing spatio-temporal databases. Additionally, they each have different sensitivities to the temporal and spatial aspects of trajectories, providing a more comprehensive evaluation of the quality of the learned trajectory representations. The specific details of the experiments is as follows:

- *Trajectory Classification (CLA).* For the Porto dataset, the driver ID is used as the label for the multi-class classification task. For the BJ dataset, whether the taxi is carrying passengers is used as the label for the binary classification task. Our TrajRL and all the baselines are fine-tuned with the loss function (Eq. (20)).
- *Travel Time Estimation (TTE).* To prevent temporal leakage, we ensured that no temporal information was fed into the model during fine-tuning, except for the departure time (timestamp of the first road segment in a trajectory). Our TrajRL and all the baselines are fine-tuned with the loss function (Eq. (21)).
- *Most Similar Trajectory Search (STS).* Similarity measure between trajectories is a fundamental research problem and serves as the basis for many trajectory-based ap-

Authorized licensed use limited to: Ocean University of China. Downloaded on September 27,2026 at 02:16:00 UTC from IEEE Xplore. Restrictions apply.

plications. Trajectory similarity measurement methods based on learning typically use the most similar trajectory search task to evaluate the effectiveness and validity of the models. However, the lack of ground truth makes it difficult to evaluate the accuracy of trajectory similarity. Following previous works [21] [43], we use the detour method [19] to generate ground truth. For specific implementation, please refer to [19].

3) Evaluation Metrics: For the trajectory classification task, we use Accuracy (ACC), F1-score (F1), Area Under Roc (AUC) to evaluate binary classification task, and Micro-F1, Macro-F1, Recall@5 to evaluate multi-classification task. For the travel time estimation task, we use Mean Absolute Error (MAE), Mean Absolute Percentage Error (MAPE), and Root Mean Square Error (RMSE) to evaluate the regression results. We use Mean Rank (MR) and Hit Ratio (HR@1, HR@5) for the most similar trajectory search task.

D. Performance Comparison

Tables III and IV show a comprehensive comparison of the performance of different trajectory representation learning methods across the three tasks. Based on the previous analysis, we believe that the quality of the learned trajectory representations is primarily related to two aspects of the model's capabilities: the capability of spatio-temporal modeling and the capability of self-supervised learning.

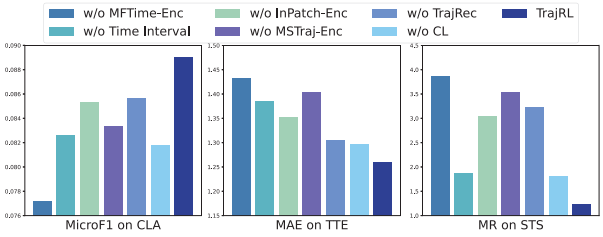
1) The capability of spatio-temporal modeling: Traj2vec, T2vec, and TrajCL are designed to model the spatio-temporal features of trajectory based on GPS coordinates, but they do not consider the semantic information in the underlying road networks. The experimental results indicate that they perform poorly on tasks related to road segments or travel semantics, such as the CLA task, which is highly dependent on the road segments traversed by the trajectories, not just their shape. Additionally, although TrajCL is the state-of-the-art model for trajectory similarity measurement, its performance in travel time estimation tasks is significantly worse due to its lack of temporal information encoding.

PIM, Trembr, Toast, JCLRNT, LightPath and START incorporate semantic information by mapping trajectory to road networks and using trajectory encoders that encode road-network constrained trajectory into embeddings. The experimental results show that, overall, they perform better on downstream tasks compared to methods that utilize GPS-based trajectories. Trembr further incorporates temporal information by using the timestamps of trajectory points as input features, significantly improving performance on travel time estimation tasks. Toast employs an enhanced method for learning road segment representations, incorporating richer semantic information. JCLRNT employs graph neural networks to learn segment information from the road network's topological structure and uses sequence models to obtain trajectory representations. However, it suffers from insufficient incorporation of road network features and lacks modeling of temporal information in trajectories. LightPath still relies on node2vec [51] for obtaining road segment embeddings, which affects its performance on downstream tasks. START simultaneously encodes static attributes of road segments as well as time periodicity, giving it significantly stronger spatio-temporal modeling capabilities and achieving experimental improvements compared to previous methods.

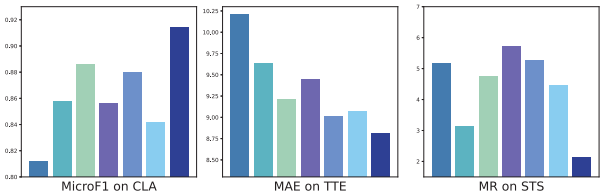
In contrast, the multifaceted time encoder and multi-scale trajectory encoder in TrajRL further enhance spatio-temporal modeling capabilities, leading to the best performance across multiple downstream tasks.

2) The capability of self-supervised learning: Traj2vec, T2vec, Trembr and LightPath all adopt the auto-encoding or auto-regressive frameworks, which focus mainly on recovering the detailed raw features of trajectory. Additionally, LightPath incorporates causal learning to enhance training effectiveness. The experimental results also show that they perform well on road segment-level downstream tasks. PIM, TrajCL, and JCLRMT use contrastive learning for pre-training. Contrastive learning enhances model performance on sequence-level tasks by learning from different views of the trajectories. Toast uses MLM as a self-supervised task which helps the model better understand the correlations between road segments in a trajectory. Our TrajRL and START employ both trajectory recovery and contrastive learning for self-supervised training, making the model optimize at both the road segment level and the sequence level simultaneously. Additionally, since these two self-supervised tasks provide signals at different scales, our multi-scale trajectory encoder can better leverage the advantages of pre-training, which also enhances our model's performance on downstream tasks. This also demonstrates the necessity of multi-scale trajectory modeling.

The experimental results and the analysis above demonstrate that our TrajRL is an effective TRL method. It possesses stronger spatio-temporal modeling and self-supervised learning capabilities, achieving the best performance across two datasets and three downstream tasks, with significant improvements compared to the state-of-the-art model.



(a) Porto Dataset



(b) BJ Dataset

Fig. 2: Results of Ablation Study.

Authorized licensed use limited to: Ocean University of China. Downloaded on September 27,2026 at 02:16:00 UTC from IEEE Xplore. Restrictions apply.

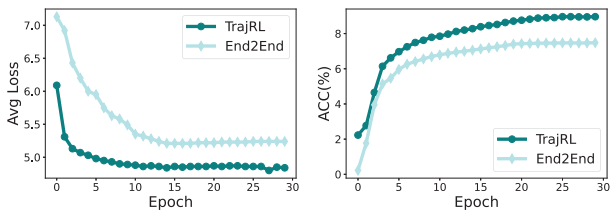
E. Ablation Study

To evaluate the effectiveness of each module of our TrajRL, we compare the performance of the following variants and the complete TrajRL: (1) **w/o MFTime-Enc**: this variant removes the whole **Multi-faceted Time Encoder**. (2) **w/o Time Interval**: this variant removes time interval embedding. (3) **w/o MSTraj-Enc**: this variant replaces the whole multi-scale trajectory encoder with a transformer encoder. (4) **w/o InPatch-Enc**: this variant replaces the InPatch encoder with a linear layer. (5) **w/o TrajRec**: this variant removes the trajectory recovery self-supervised task. (6) **w/o CL**: this variant removes the contrastive learning self-supervised task.

Additionally, we investigate the effectiveness of pre-training in the next section, which involves a variant that removes all self-supervised tasks.

1) Impact of model components: The experimental results in Figure 2 validate the effectiveness of each module on both two datasets. First, **w/o MFTime-Enc** significantly reduces performance, which demonstrates the necessity of temporal encoding in TRL. Furthermore, the comparison between **w/o Time Interval** and **TrajRL** demonstrates that the multi-scale time interval encoding benefits multiple downstream tasks, particularly having a significant impact on travel time estimation. Therefore, multi-scale temporal interval coding is very meaningful for enriching time coding. In addition, benefiting from multi-scale feature learning capabilities, TrajRL significantly performs better than **w/o MSTraj-Enc** which uses a transformer encoder. This advantage is more prominent in the STS task. TrajRL using GRU to aggregate information from a patch's sub-trajectory segments is superior to **w/o InPatch-Enc** using a linear layer. Therefore, we believe that further exploring short sequence encoders that are more suitable for the InPatch encoder can help improve model performance.

2) Impact of Self-Supervised Tasks: The results indicate that both self-supervised tasks significantly affect the model's performance on the whole. Specifically, The performance of **w/o TrajRec** and **w/o CL** drops even more on the CLA and STS than on TTE. This is because trajectory recovery and contrastive learning provide a lot of spatial self-supervised signals, but less temporal self-supervision information. Furthermore, **w/o CL** has the largest performance drop on CLA tasks, while **w/o TrajRec** has the largest performance drop on the STS task. Considering the different scales of these two tasks, this further verifies the necessity of providing self-supervised signals for TRL in different scales through different self-supervised tasks.



(a) Average loss (b) Validation accuracy

Fig. 3: Pre-training+Fine-tuning vs. End2End training.

F. Effectiveness of Pre-training

To explore the pre-training effects of the model, we compare the complete TrajRL with a variant that differs in their training methods. The end-to-end training method randomly initializes the model parameters and directly trains the model using the loss function of the downstream task for supervision. Additionally, we investigate the model's transferability across different datasets. We pre-train the model on one dataset and fine-tune it on another dataset.

TABLE V: Model transferability across two datasets.

Dataset	Model	Trajectory Classification		Travel Time Estimation	
		Micro-F1↑	Macro-F1↑	MAE ↓	RMSE↓
Porto	START	0.084	0.061	1.374	2.185
	TrajRL	0.092	0.070	1.259	2.051
BJ→ Porto	START	0.076	0.055	1.444	2.385
	TrajRL	0.081	0.057	1.346	2.153
		ACC ↑	F1 ↑	MAE ↓	RMSE↓
BJ	START	0.842	0.881	9.134	35.31
	TrajRL	0.867	0.914	8.807	33.15
Porto→BJ	START	0.807	0.824	11.29	42.21
	TrajRL	0.826	0.827	10.71	40.98

The average loss values and validation accuracy for CLA on the Porto dataset are reported in Figure 3. The results indicate that pre-training and fine-tuning can significantly improve model's convergence speed. Additionally, pre-training enhances the model's generalization ability on downstream tasks, owing to the prior knowledge incorporated in the pre-training tasks.

In Table V, we report the cross-dataset transfer performance of our TrajRLand START. Experimental results show that TrajRL achieves about 80% of the performance of the model trained directly on the target dataset. Compared to the state-of-the-art model START, TrajRL also demonstrates stronger cross-dataset transfer capabilities. Although the road networks in different cities are heterogeneous, our TrajRL can capture common travel patterns. However, capturing city-specific driving habits remains challenging, which limits the transferability of trajectory representation across different datasets. The results also indicate that TrajRL demonstrates good robustness, primarily due to the appropriate pretraining methods.

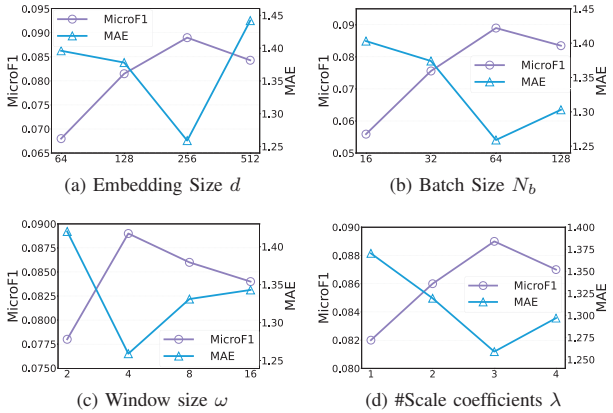
G. Impact of Hyper-parameters

We further conduct the parameter sensitivity analysis for critical hyper-parameters, *i.e.*, embedding size d, batch size N_b, the window size ω in the InPatch encoder, and the number of scale coefficients λ in the time interval encoder. We report the results of CLA (by MicroF1) and TTE (by MAE) tasks on Porto in Figure 4, and the results on BJ are similar.

From Figure 4(a), we can see that increasing the embedding size d initially improves the model performance but when it becomes too large, the performance deteriorates due to sparse learned trajectory embeddings or model overfitting.

A large batch size is more likely to help the model converge, but if it becomes too large, it negatively impacts the effectiveness of contrastive learning, leading to a decline in model

Authorized licensed use limited to: Ocean University of China. Downloaded on September 27,2026 at 02:16:00 UTC from IEEE Xplore. Restrictions apply.



(a) Embedding Size d (b) Batch Size N_b

(c) Window size ω (d) #Scale coefficients λ

Fig. 4: Impact of Hyper-parameters on CLA and TTE on Porto.

performance. The results *w.r.t.* batch size N_b in Figure 4(b) verify this theoretical analysis.

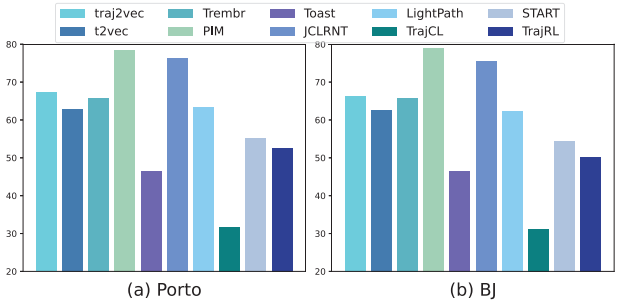
The window size determines the scale of each layer and the number of scaled representations in the multi-scale trajectory encoder. A window size that is too small hinders the model from learning fine-grained trajectory representations, while an over-large window size results in the model learning fewer scaled representations. Experimental results in Figure 4(c) indicate that the model performs best when the window size ω is set to 4.

Additionally, we study the impact of the scale coefficients in the multi-faceted time encoder on model performance. Given that our TTE task is measured in minutes, we prioritize incorporating 60 (seconds) into our choice of scale coefficients. Therefore, we experiment with four different scale coefficients *i.e.*, $\{60\}$, $\{30, 60\}$, $\{10, 30, 60\}$, and $\{10, 30, 60, 90\}$. From Figure 4(d), using more scale coefficients significantly improves the model's performance, as a greater number of scale coefficients allows for finer granularity in time interval encoding. Ideally, the more scale coefficients there are, the better the model's performance. However, in practice, time intervals between road segments in a trajectory are not excessively large, so overly large scale coefficients are ineffective and may introduce noise. Additionally, the travel time estimation task is particularly affected by the choice of scale coefficients, further validating the necessity of time interval encoding.

H. Efficiency Comparison

We also compare the efficiency of our TrajRL with all baselines on the two datasets. Since our TrajRL is a self-supervised pre-trained model that can be easily trained offline, we primarily focus on the time cost of encoding trajectories into embeddings. We report the time cost of encoding 200,000 trajectories in Figure 5.

Overall, the inference speed of our TrajRL is at a mid-range level. More importantly, except for TrajCL [28], which is specifically designed for rapid trajectory queries, our TrajRL exhibits a relatively advanced inference speed among general



(a) Porto (b) BJ

Fig. 5: Inference Time Comparison (Second).

trajectory representation learning methods. Our model takes about 50 seconds to encode 200,000 trajectories, demonstrating its scalability to large trajectory datasets. Compared to the currently state-of-the-art TRL method, *i.e.*, START, TrajRL also shows some advantages. Notably, although our multi-scale trajectory encoder is a deeper model, the time complexity analysis in Section IV-F and experimental results both indicate that the inference speed of TrajRL does not deteriorate compared to other self-attention based models. This is primarily due to two key advantages of our multi-scale trajectory encoder. First, self-attention calculations in the CrossPatch encoder are performed on patches rather than points, which significantly reduces the time complexity. Second, since the InPatch encoder merges patches within a window, the length of input sequence and the number of parameters in deeper scale blocks decreases exponentially compared to shallower ones. These two advantages greatly facilitate the construction and training of deeper models. In summary, our TrajRL achieves deeper architecture with lower time cost and exhibits strong scalability.

VI. Conclusion and Future Work

In this work, we propose TrajRL, a novel self-supervised trajectory representation learning framework. TrajRL effectively learns general trajectory embeddings by the designed multi-faceted time encoder and multi-scale trajectory encoder. The multi-faceted time encoder models time periods and time intervals simultaneously, capturing comprehensive temporal features of trajectories. The multi-scale trajectory encoder learns the multi-scale spatio-temporal features of trajectories through InPatch and CrossPatch encoders. Experiments on two real-world datasets across three downstream tasks demonstrate the superiority of our proposed TrajRL against state-of-the-art baselines. In the future, we will explore a unified representation learning model for large-scale multi-source heterogeneous trajectories combined with large language models.

VII. Acknowledgment

This work is partially supported by the National Natural Science Foundation of China under Grant Nos. 62176243 and 62202438, and the National Key R&D Program of China under Grant No 2022ZD0117201.

Authorized licensed use limited to: Ocean University of China. Downloaded on September 27,2026 at 02:16:00 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] L. Chen, Y. Gao, X. Li, C. S. Jensen, and G. Chen, "Efficient metric indexing for similarity search and similarity joins," *IEEE Transactions on Knowledge and Data Engineering*, vol. 29, no. 3, pp. 556–571, 2017.

[2] X. Ding, L. Chen, Y. Gao, C. S. Jensen, and H. Bao, "Ultraman: A unified platform for big trajectory data management and analytics," *Proceedings of the VLDB Endowment*, vol. 11, no. 7, pp. 787–799, 2018.

[3] J. Bao, T. He, S. Ruan, Y. Li, and Y. Zheng, "Planning bike lanes based on sharing-bikes' trajectories," in *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, 2017, pp. 1377–1386.

[4] L. Chen, Q. Zhong, X. Xiao, Y. Gao, P. Jin, and C. S. Jensen, "Price-and-time-aware dynamic ridesharing," in *2018 IEEE 34th international conference on data engineering (ICDE)*. IEEE, 2018, pp. 1061–1072.

[5] W. Chen, S. Li, C. Huang, Y. Yu, Y. Jiang, and J. Dong, "Mutual distillation learning network for trajectory-user linking," *arXiv preprint arXiv:2205.03773*, 2022.

[6] S. Dai, Y. Yu, H. Fan, and J. Dong, "Spatio-temporal representation learning with social tie for personalized poi recommendation," *Data Science and Engineering*, vol. 7, no. 1, pp. 44–56, 2022.

[7] T. Feng, H. Yan, H. Wang, W. Huang, Y. Han, H. Liao, J. Hao, and Y. Li, "Ilroute: A graph-based imitation learning method to unveil riders' routing strategies in food delivery service," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, pp. 4024–4034.

[8] Y. Liang, K. Ouyang, Y. Wang, X. Liu, H. Chen, J. Zhang, Y. Zheng, and R. Zimmermann, "Trajformer: Efficient trajectory classification with transformers," in *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, 2022, pp. 1229–1237.

[9] Z. Wang, K. Fu, and J. Ye, "Learning to estimate the travel time," in *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2018, pp. 858–866.

[10] D. Yao, G. Cong, C. Zhang, and J. Bi, "Computing trajectory similarity in linear time: A generic seed-guided neural metric learning approach," in *2019 IEEE 35th international conference on data engineering (ICDE)*. IEEE, 2019, pp. 1358–1369.

[11] L. Chen, Y. Gao, Z. Fang, X. Miao, C. S. Jensen, and C. Guo, "Real-time distributed co-movement pattern detection on streaming trajectories," *Proceedings of the VLDB Endowment*, vol. 12, no. 10, pp. 1208–1220, 2019.

[12] L. Alessandretti, "What human mobility data tell us about covid-19 spread," *Nature Reviews Physics*, vol. 4, no. 1, pp. 12–13, 2022.

[13] H. Liu, H. Wu, W. Sun, and I. Lee, "Spatio-temporal gru for trajectory classification," in *2019 IEEE International Conference on Data Mining (ICDM)*. IEEE, 2019, pp. 1228–1233.

[14] X. Li, K. Zhao, G. Cong, C. S. Jensen, and W. Wei, "Deep representation learning for trajectory similarity computation," in *2018 IEEE 34th international conference on data engineering (ICDE)*. IEEE, 2018, pp. 617–628.

[15] A. Ashukha, A. Lyzhov, D. Molchanov, and D. Vetrov, "Pitfalls of in-domain uncertainty estimation and ensembling in deep learning," *arXiv preprint arXiv:2002.06470*, 2020.

[16] T. Ishida, I. Yamane, T. Sakai, G. Niu, and M. Sugiyama, "Do we need zero training loss after achieving zero training error?" *arXiv preprint arXiv:2002.08709*, 2020.

[17] Z. Fang, Y. Du, L. Chen, Y. Hu, Y. Gao, and G. Chen, "E2dtc: An end to end deep trajectory clustering framework via self-training," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 2021, pp. 696–707.

[18] D. Yao, C. Zhang, Z. Zhu, J. Huang, and J. Bi, "Trajectory clustering via deep representation learning," in *2017 international joint conference on neural networks (IJCNN)*. IEEE, 2017, pp. 3880–3887.

[19] Y. Chen, X. Li, G. Cong, Z. Bao, C. Long, Y. Liu, A. K. Chandran, and R. Ellison, "Robust road network representation learning: When traffic patterns meet traveling semantics," in *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, 2021, pp. 211–220.

[20] S. B. Yang, C. Guo, J. Hu, J. Tang, and B. Yang, "Unsupervised path representation learning with curriculum negative sampling," in *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*, 2021, pp. 3286–3292.

[21] J. Jiang, D. Pan, H. Ren, X. Jiang, C. Li, and J. Wang, "Self-supervised trajectory representation learning with temporal regularities and travel semantics," in *2023 IEEE 39th international conference on data engineering (ICDE)*. IEEE, 2023, pp. 843–855.

[22] T.-Y. Fu and W.-C. Lee, "Trembr: Exploring road networks for trajectory representation learning," *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 11, no. 1, pp. 1–25, 2020.

[23] D. Yao, H. Hu, L. Du, G. Cong, S. Han, and J. Bi, "Trajgat: A graph-based long-term dependency modeling approach for trajectory similarity computation," in *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*, 2022, pp. 2275–2285.

[24] S. Zhou, J. Li, H. Wang, S. Shang, and P. Han, "Grlstm: trajectory similarity computation with graph-based residual lstm," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 4, 2023, pp. 4972–4980.

[25] A. Liu, Y. Zhang, X. Zhang, G. Liu, Y. Zhang, Z. Li, L. Zhao, Q. Li, and X. Zhou, "Representation learning with multi-level attention for activity trajectory similarity computation," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 5, pp. 2387–2400, 2020.

[26] L. Deng, Y. Zhao, Z. Fu, H. Sun, S. Liu, and K. Zheng, "Efficient trajectory similarity computation with contrastive learning," in *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, 2022, pp. 365–374.

[27] Y. Cao, L. Li, X. Chen, X. Xu, Z. Huang, and Y. Yu, "Hypergraph hash learning for efficient trajectory similarity computation," in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, 2024, pp. 175–186.

[28] Y. Chang, J. Qi, Y. Liang, and E. Tanin, "Contrastive trajectory similarity learning with dual-feature attention," in *2023 IEEE 39th International conference on data engineering (ICDE)*. IEEE, 2023, pp. 2933–2945.

[29] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," *Advances in neural information processing systems*, vol. 26, 2013.

[30] Z. Mao, Z. Li, D. Li, L. Bai, and R. Zhao, "Jointly contrastive representation learning on road network and trajectory," in *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, 2022, pp. 1501–1510.

[31] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," *arXiv preprint arXiv:1710.10903*, 2017.

[32] S. B. Yang, J. Hu, C. Guo, B. Yang, and C. S. Jensen, "Lightpath: Lightweight and scalable path representation learning," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, pp. 2999–3010.

[33] T. Wei, Y. Lin, Y. Lin, S. Guo, L. Zhang, and H. Wan, "Micro-macro spatial-temporal graph-based encoder-decoder for map-constrained trajectory recovery," *IEEE Transactions on Knowledge and Data Engineering*, 2024.

[34] C. Feng, Z. Pan, J. Fang, J. Xu, P. Zhao, and L. Zhao, "Aries: Accurate metric-based representation learning for fast top-k trajectory similarity query," in *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, 2022, pp. 499–508.

[35] P. Yang, H. Wang, Y. Zhang, L. Qin, W. Zhang, and X. Lin, "T3s: Effective representation learning for trajectory similarity computation," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 2021, pp. 2183–2188.

[36] H. Zhang, X. Zhang, Q. Jiang, B. Zheng, Z. Sun, W. Sun, and C. Wang, "Trajectory similarity learning with auxiliary supervision and optimal matching," in *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*, 2021.

[37] S. B. Yang, C. Guo, J. Hu, B. Yang, J. Tang, and C. S. Jensen, "Weakly-supervised temporal path representation learning with contrastive curriculum learning," in *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 2022, pp. 2873–2885.

[38] B. Yan, G. Zhao, L. Song, Y. Yu, and J. Dong, "Precln: Pretrained-based contrastive learning network for vehicle trajectory prediction," *World Wide Web*, vol. 26, no. 4, pp. 1853–1875, 2023.

[39] K. Luo, Y. Zhu, W. Chen, K. Wang, Z. Zhou, S. Ruan, and Y. Liang, "Towards robust trajectory representations: Isolating environmental confounders with causal learning," *arXiv preprint arXiv:2404.14073*, 2024.

[40] X. Qiu, T. Sun, Y. Xu, Y. Shao, N. Dai, and X. Huang, "Pre-trained models for natural language processing: A survey," *Science China technological sciences*, vol. 63, no. 10, pp. 1872–1897, 2020.

Authorized licensed use limited to: Ocean University of China. Downloaded on September 27,2026 at 02:16:00 UTC from IEEE Xplore. Restrictions apply.

[41] J. Yu, H. Yin, X. Xia, T. Chen, J. Li, and Z. Huang, "Self-supervised learning for recommender systems: A survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 1, pp. 335–355, 2023.
[42] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
[43] Y. Lin, H. Wan, S. Guo, J. Hu, C. S. Jensen, and Y. Lin, "Pre-training general trajectory embeddings with maximum multi-view entropy coding," *IEEE Transactions on Knowledge and Data Engineering*, 2023.
[44] C. Yang and G. Gidofalvi, "Fast map matching, an algorithm integrating hidden markov model with precomputation," *International Journal of Geographical Information Science*, vol. 32, no. 3, pp. 547–570, 2018.
[45] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.
[46] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," *arXiv preprint arXiv:2211.14730*, 2022.
[47] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *arXiv preprint arXiv:1412.3555*, 2014.
[48] F. Wang and H. Liu, "Understanding the behaviour of contrastive loss," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 2495–2504.
[49] J. Jiang, C. Han, W. Jiang, W. X. Zhao, and J. Wang, "Libcity: A unified library towards efficient and comprehensive urban spatial-temporal prediction," *arXiv preprint arXiv:2304.14343*, 2023.
[50] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019.
[51] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, 2016, pp. 855–864.

Authorized licensed use limited to: Ocean University of China. Downloaded on September 27,2026 at 02:16:00 UTC from IEEE Xplore. Restrictions apply.